



Programare orientată pe obiecte

- suport de curs -

Andrei Păun
Anca Dobrovăț

An universitar 2025 – 2026
Semestrul II
Seriile 13, 14, 15

Curs 4

Cuprins

- Recapitulare curs 3
- Membrii statici ai unei clase
- Clase locale (în funcții și în alte clase)
- Operatorul ::
- supraîncărcarea funcțiilor în C++
- supraîncărcarea operatorilor în C++

Variabile locale (reminder)

- **declarate în interiorul unei funcții sau al unui bloc de cod** ({ ... })
- domeniul de vizibilitate (scope): local, există **doar în interiorul aceluși bloc** și nu poate fi accesată din afara lui
- memoria este de obicei alocată pe **stack**
- **nu sunt inițializate automat**

```
1  #include <iostream>
2
3  using namespace std;
4
5  void f()
6  {
7      int x = 56;
8      x++;
9      cout<<&x<<" "<<x<<endl;
10 }
11 int main()
12 {
13     f(); f(); f();
14     return 0;
15 }
```

Se va afișa 57, 57, 57

- Adresa de memorie (pe PC-ul propriu): 0x8900fffaac – probabil o adresă pe stack (în general, adresele mari aparțin stack-ului)

Variabile locale statice (reminder)

- **declarate în interiorul unei funcții sau al unui bloc de cod** ({ ... }) dar folosind “**static**”
- domeniul de vizibilitate (scope): local, există **doar în interiorul aceluiași bloc** și nu poate fi accesată din afara lui
- **durată de viață statică** – există pe toată durata programului

```
1  #include <iostream>
2
3  using namespace std;
4
5  void f()
6  {
7      static int x = 56;
8      x++;
9      cout<<&x<<" "<<x<<endl;
10 }
11 int main()
12 {
13     f(); f(); f();
14     return 0;
15 }
```

Se va afișa 57, 58, 59

- Adresa de memorie (pe PC-ul propriu): 0x7ff785424010

Membrii statici ai unei clase

- date membre:
 - nestatice (distincte pentru fiecare obiect);
 - **static** (unice pentru toate obiectele clasei, există o singură copie pentru toate obiectele).
- cuvânt cheie “**static**”
- create, inițializate și accesate – independent de obiectele clasei.
- Variabile - alocarea și inițializarea – în afara clasei.
- Const – alocarea și inițializarea – în interiorul clasei

Membrii statici ai unei clase

- Variabile - alocarea și inițializarea – în afara clasei.
- Const – alocarea și inițializarea – în interiorul clasei

```
1  #include <iostream>
2
3  using namespace std;
4
5  class Test
6  {
7      static int x;
8  };
9  int Test::x;
10 int main()
11 {
12     cout<<sizeof(Test);
13 }
```

```
1  #include <iostream>
2
3  using namespace std;
4
5  class Test
6  {
7      const static int x = 0;
8  };
9
10 int main()
11 {
12     cout<<sizeof(Test);
13 }
```

Membrii statici ai unei clase

- create, inițializate și accesate – independent de obiectele clasei.

```
1  #include <iostream>
2
3  using namespace std;
4
5  class Test
6  {
7      static int x;
8  };
9  int Test::x;
10 int main()
11 {
12     cout<<sizeof(Test);
13 }
```

```
1  #include <iostream>
2
3  using namespace std;
4
5  class Test
6  {
7      const static int x = 0;
8  };
9
10 int main()
11 {
12     cout<<sizeof(Test);
13 }
```

```
1  #include <iostream>
2
3  using namespace std;
4
5  class Test
6  {
7      int x;
8  };
9
10 int main()
11 {
12     cout<<sizeof(Test);
13 }
```

- Se afișează 1 (dimensiunea “onorifică” a unei clase fără date membre nestatice).

- Se afișează 4 (dimensiunea unui int – dată membră nestatică x).

Membrii statici ai unei clase

- funcțiile statice:
 - efectuează operații asupra întregii clase;
 - nu au cuvântul cheie “this”;
 - se pot referi doar la membrii statici.
- referirea membrilor statici:
 - `clasa :: membru`;
 - `obiect.membru` (identic cu `nestatic`).

Membrii statici ai unei clase

O funcție membră statică:

- poate fi apelată înainte de crearea obiectelor;
- poate referi membri statici
- poate fi apelată și ca funcțiile membre nestatice prin `obiect.nume_funcție`
- nu poate referi, în mod direct, date sau funcții membre nestatice
- poate referi, în mod indirect, prin intermediul unui obiect local, date sau funcții membre nestatice
- nu au pointerul **this**

```
class Test
{
    static int x;
    int y;
public:
    void functie_nestatica() {}
    static void f1() {x = 1;}
    static void f2() {
        y = 2;
        functie_nestatica();
    }

    static void f3(Test o) {o.x = 3; o.y = 4;}
    static void f4() {Test o; o.x = 5; o.y = 6;}
    static void f5() {this->x = 7;}
};

int Test::x;
```

```
int main()
{
    Test::f1();
    Test ob;
    ob.f1();
    Test::f2();
    Test::f3(ob);
    Test::f4();
    Test::f5();
    return 0;
}
```

Obiecte statice

- Precedate de cuvântul **static**
- Se inițializează o singură dată și se utilizează dacă se dorește păstrarea valorii la apelul multiplu al funcției
- 2 tipuri: - obiecte **locale** statice și obiecte **globale** statice
- Un obiect static local este distrus la finalul programului, dar, este vizibil doar în scope-ul în care a fost definit.
- Obiecte globale statice sunt ultimele distruse

Obiecte statice locale

Un obiect static local este distrus la finalul programului.

```
#include <iostream>

using namespace std;

class Test{ int a;
public:
    Test(int x = 0):a(x){cout<<"C " <<a<<endl;}
    ~Test(){cout<<"D " <<a<<endl;}
};

void f(int i){ static Test obiect(i); }

int main(){
    cout<<"start"<<'\n';
    f(1);
    cout<<"end"<<'\n';
}
```

```
start
C 1
end
D 1
```

Obiecte statice locale

Ce se întâmplă dacă se apelează funcția de mai multe ori?

```
#include <iostream>

using namespace std;

class Test{ int a;
public:
    Test(int x = 0):a(x){cout<<"C " <<a<<endl;}
    ~Test() {cout<<"D " <<a<<endl;}
};

void f(int i){ static Test obiect(i); cout<<&obiect<<endl;}

int main()
{
    cout<<"start"<<"\n";
    f(1);
    f(2);
    f(3);
    cout<<"end"<<"\n";
}
```

```
start
C 1
0x407034
0x407034
0x407034
end
D 1
```

De câte ori este creat și distrus obiectul static local? Răspuns: o singură dată

Obiecte statice locale (în funcții membru)

```
class Test
{
    int x;
public:
    Test(int x=0): x(x) {cout<<x<<" C\n";}
    ~Test() {cout<<x<<" D\n";}
    void f()
    {
        static Test obiect;
    }
};

int main()
{
    Test obiect1(1);
    cout<<"start"<<"\n";
    obiect1.f();
    cout<<"end"<<"\n";
}
```

```
1 C
start
0 C
end
1 D
0 D
```

De câte ori este creat și distrus obiectul static local? Răspuns: o singură dată

Obiecte statice globale (sunt distruse ultimele)

```
class Test
{
    int x;
public:
    Test(int x=0): x(x) {cout<<x<<" C\n";}
    ~Test() {cout<<x<<" D\n";}
    void f() { static Test obiect; cout<<&obiect<<endl; }
};

static Test A(2), B(3);

int main()
{
    Test obiect1(1);
    cout<<"start"<<'\\n';
    obiect1.f();
    obiect1.f();
    cout<<"end"<<'\\n';
}
```

```
2 C
3 C
1 C
start
0 C
0x4030c0
0x4030c0
end
1 D
0 D
3 D
2 D
```

Operatorul de rezolutie de scop ::

```
int i; // global i
```

```
void f()
```

```
{
```

```
    int i; // local i
```

```
    i = 10; // uses local i.
```

```
}
```

```
int i; // global i
```

```
void f()
```

```
{
```

```
    int i = 7; // local i
```

```
    ::i = 10; // now refers to global i
```

```
    Cout<<::i;
```

```
}
```

Clase locale

- putem defini **clase în clase** sau **în funcții**
- **class** este o declarație, deci definește un scop
- operatorul de rezoluție de scop ajută în aceste cazuri
- rar utilizate clase în clase

Clase locale (în funcții)

```
void f();  
int main() {  
    f(); /// myclass not known here  
    return 0; }  
void f() {  
    class myclass  
    {  
        int i;  
    public:  
        void put_i(int n) { i=n; }  
        int get_i() { return i; }  
    } ob;  
    ob.put_i(10);  
    cout << ob.get_i();  
}
```

Funcțiile membre:

- trebuie definite în clasă
- nu pot accesa variabilele locale ale funcției
- pot accesa variabilele locale statice
- pot accesa variabilele globale

Clasele locale:

- Nu pot avea variabile statice, dar pot folosi funcții statice
- Pot fi puse în legătură cu alte clase locale din aceeași funcție

! Mai multe exemple în fisierul cpp încărcat în Moodle

Clase locale (în alte clase) – imbricate (“nested”)

```
int global = 300;
class Out{
public:
    class In{
        static int A;
        int x;
    public:
        void set(int i);
        int get(){return A + x + global;}
    }ob2;
};

int Out::In::A = 90;
void Out::In::set(int i){ x = i;}

int main()
{
    Out ob;
    ob.ob2.set(100);
    cout<<ob.ob2.get();
    return 0;
}
```

- Funcțiile membre ale clasei locale trebuie declarate și definite în clasa interioară sau în afara clasei exterioare
- Pot avea date membre statice
- Pot accesa variabile globale

!atenție “public”: doar dpv didactic/demonstrativ - pt a avea acces direct la ob2 din main

Funcții care întorc obiecte

- o funcție poate întoarce obiecte
- un obiect temporar este creat automat pentru a ține informațiile din obiectul de întors
- acesta este obiectul care este întors
- după ce valoarea a fost întoarsă, acest obiect este distrus
- probleme cu memoria dinamică: soluție
polimorfism pe = și pe constructorul de copiere

// Returning objects from a function.

```
#include <iostream>
```

```
using namespace std;
```

```
class myclass
```

```
{
```

```
    int i;
```

```
public:
```

```
myclass(){ i=0;}
```

```
    void set_i(int n) { i=n; }
```

```
    int get_i() { return i; }
```

```
};
```

```
myclass f(); // return object of type myclass
```

```
int main()
```

```
{
```

```
    myclass o;
```

```
    o = f();
```

```
    cout << o.get_i() << "\n";
```

```
    return 0;
```

```
}
```

```
myclass f()
```

```
{
```

```
    myclass x;
```

```
    x.set_i(1);
```

```
    return x;
```

```
}
```

copierea prin operatorul =

- este posibil să dăm valoarea unui obiect altui obiect
- trebuie să fie de același tip (aceeași clasă)

Supraîncărcarea operatorilor în C++

- majoritatea operatorilor pot fi supraîncărcați
- similar ca la funcții
- una din proprietățile C++ care îi conferă putere
- s-a făcut supraîncărcarea operatorilor si pentru operații de I/O (<<, >>)
- supraîncărcarea se face definind o funcție operator: membru al clasei sau nu

Restricții

- nu se poate redefini și precedența operatorilor
- nu se poate redefini numărul de operanzi
 - rezonabil pentru că redefinim pentru lizibilitate
 - putem ignora un operand dacă vrem
- nu putem avea valori implicite; excepție pentru ()
- **nu putem face overload pe**
 - . (acces de membru)**
 - :: (rezoluție de scop)**
 - .*(acces membru prin pointer)**
 - ?: (ternar)**
 - sizeof()**
 - typeid**
 - alignof (operator pentru alinierea tipurilor; din C++11)**
- Se recomandă operațiuni apropiate de înțelesul operatorilor respectivi

Funcții operator membri ai clasei

```
ret-type class-name::operator#(arg-list)  
{  
    // operations  
}
```

- # este operatorul supraîncărcat (+ - * / ++ -- = , etc.)
- de obicei *ret-type* este tipul clasei, dar avem flexibilitate
- pentru operatori unari *arg-list* este vidă
- pentru operatori binari: *arg-list* conține un element


```

class loc {
    int longitude, latitude;
public:
    loc() {}
    loc(int lg, int lt) {
        longitude = lg;
        latitude = lt; }
    void show() {
        cout << longitude << " ";
        cout << latitude << "\n";
    }
    loc operator+(loc op2);
};

```

// Overload + for loc.

```

loc loc::operator+(loc op2)
{
    loc temp;
    temp.longitude = op2.longitude + longitude;
    temp.latitude = op2.latitude + latitude;
    return temp;
}

```

```

int main(){
    loc ob1(10, 20), ob2( 5, 30);
    ob1.show(); // displays 10 20
    ob2.show(); // displays 5 30
    ob1 = ob1 + ob2;
    ob1.show(); // displays 15 50
    return 0;
}

```

- un singur argument pentru că avem **this**
- longitude==this->longitude
- obiectul din stânga face apelul la funcția operator
 - ob1a chemat operatorul + redefinit în clasa lui ob1

- **daca întoarcem același tip de date în operator putem avea expresii**
- **daca întorceam alt tip nu puteam face**
$$\text{ob1} = \text{ob1} + \text{ob2};$$
- **putem avea si**
`(ob1+ob2).show(); // displays outcome of ob1+ob2`
- **pentru ca funcția show() este definita in clasa lui ob1**
- **se generează un obiect temporar**
 - (constructor de copiere)

```

#include <iostream>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {} // needed to construct temporaries
    loc(int lg, int lt) { longitude = lg; latitude = lt; }
    void show() { cout<<longitude<<" "<<latitude<<"\n"; }
    loc operator+(loc op2);
    loc operator-(loc op2);
    loc operator=(loc op2);
    loc operator++();
};
// Overload + for loc.
loc loc::operator+(loc op2){ loc temp;
    temp.longitude = op2.longitude + longitude;
    temp.latitude = op2.latitude + latitude;
    return temp;}

loc loc::operator-(loc op2){ loc temp;
    temp.longitude = longitude - op2.longitude;
    temp.latitude = latitude - op2.latitude;
    return temp;}

```

```

// Overload assignment for loc.
loc loc::operator=(loc op2){
    longitude = op2.longitude;
    latitude = op2.latitude;
    return *this; } // object that generated call
// Overload prefix ++ for loc.
loc loc::operator++(){
    longitude++;
    latitude++;
    return *this;}

int main(){
    loc ob1(10, 20), ob2( 5, 30), ob3(90, 90);
    ob1.show(); ob2.show();
    ++ob1; ob1.show(); // displays 11 21
    ob2 = ++ob1; ob1.show(); // displays 12 22
    ob2.show(); // displays 12 22
    ob1 = ob2 = ob3; // multiple assignment
    ob1.show(); // displays 90 90
    ob2.show(); // displays 90 90
    return 0;}

```

- apelul la funcția operator se face din obiectul din stânga (pentru operatori binari)
 - din aceasta cauza pentru – avem funcția definita asa
- operatorul = face copiere pe variabilele de instanță, întoarce *this
- se pot face atribuiri multiple (dreapta spre stânga)

Formele prefix și postfix

- am văzut prefix, pentru postfix: definim un parametru int “dummy”

```
// Prefix increment  
type operator++( ) {  
    // body of prefix operator  
}
```

```
// Postfix increment  
type operator++( int x) {  
    // body of postfix operator  
}
```

Supraîncărcarea +=, *=, etc.

```
loc loc::operator+=(loc op2)
{
    longitude = op2.longitude + longitude;
    latitude = op2.latitude + latitude;
    return *this;
}
```

- Este posibil să facem o decuplare completă între înțelesul inițial al operatorului
 - exemplu: << >>
- moștenire: operatorii (mai puțin =) sunt moșteniți de clasă derivată
- clasă derivată poate să își redefinească operatorii

Supraîncărcarea operatorilor ca funcții prieten

- operatorii pot fi definiți și ca funcție nemembră a clasei
- o facem funcție prietenă pentru a putea accesa rapid câmpurile protejate
- **nu avem pointerul “this”**
- **deci vom avea nevoie de toți operanzii** ca parametri pentru funcția operator
- primul parametru este operandul din stânga, al doilea parametru este operandul din dreapta


```

#include <iostream>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {} // needed to construct temporaries
    loc(int lg, int lt) { longitude = lg; latitude = lt; }
    void show() { cout<<longitude<<" "<<latitude<<"\n";}
    friend loc operator+(loc op1, loc op2); // friend
    loc operator-(loc op2);
    loc operator=(loc op2);
    loc operator++();
};

```

// Now, + is overloaded using friend function.

```

    loc operator+(loc op1, loc op2){
        loc temp;
        temp.longitude = op1.longitude + op2.longitude;
        temp.latitude = op1.latitude + op2.latitude;
        return temp;
    }

```

```

loc loc::operator-(loc op2){ loc temp;
// notice order of operands
    temp.longitude = longitude - op2.longitude;
    temp.latitude = latitude - op2.latitude;
    return temp;}

```

// Overload asignment for loc.

```

loc loc::operator=(loc op2){
    longitude = op2.longitude;
    latitude = op2.latitude;
    return *this; }// object that generated call

```

```

loc loc::operator++(){
    longitude++;
    latitude++;
    return *this;}

```

```

int main(){
    loc ob1(10, 20), ob2( 5, 30);
    ob1 = ob1 + ob2;
    ob1.show();
    return 0;}

```

Restricții pentru operatorii definiți ca prieten

- **nu se pot supraîncărca utilizând funcții prieten:**
 - Operatorul de atribuire =
 - Operatorul apel de funcție ()
 - Operatorul de indexare []
 - Operatorul de acces prin pointer la funcții membre ->
- pentru ++ sau -- trebuie să folosim referințe

Funcții prieten pentru operatori unari

- pentru ++, -- folosim referința pentru a transmite operandul
 - pentru că trebuie să se modifice și nu avem pointerul this
 - apel prin valoare: primim o copie a obiectului și nu putem modifica operandul (ci doar copia)

```

#include <iostream>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {} // needed to construct temporaries
    loc(int lg, int lt) { longitude = lg; latitude = lt; }
    void show() { cout<<longitude<<" "<<latitude<<"\n";}
    loc operator=(loc op2);
    friend loc operator++(loc& op);
    friend loc operator--(loc& op);
};
// Overload assignment for loc.
loc loc::operator=(loc op2){
    longitude = op2.longitude;
    latitude = op2.latitude;
    return *this; }// object that generated call

// Now a friend, use a reference parameter.
    loc operator++(loc& op) {
        op.longitude++;
        op.latitude++;
    return op;
}

```

```

// Make – a friend. Use reference
    loc operator--(loc& op) {
        op.longitude--;
        op.latitude--;
    return op;
}

int main(){
    loc ob1(10, 20), ob2;
    ob1.show();
    ++ob1;
    ob1.show(); // displays 11 21
    ob2 = ++ob1;
    ob2.show(); // displays 12 22
    --ob2;
    ob2.show(); // displays 11 21
    return 0;}

```

pentru varianta postfix ++ --

- la fel ca la supraîncărcarea operatorilor prin funcții membru ale clasei: parametru int

```
// friend, postfix version of ++  
friend loc operator++(loc &op, int x);
```

Diferențe supraîncărcarea prin membri sau prieteni

- de multe ori nu avem diferențe,
 - atunci e indicat să folosim funcții membru
 - ca funcții membru: operatori unari, cei compuși ($+=$, $*=$ etc)
- uneori avem însă diferențe: poziția operanzilor
 - pentru funcții membru operandul din stânga apelează funcția operator supraîncărcată
 - dacă vrem să scriem expresie: `100+ob`; probleme la compilare=> funcții prieten
- Interesant: depinde de operator și de situație (sursa: <https://stackoverflow.com/questions/4421706>)

“Spaceship operator” $< = >$ in C++20

- Se supraîncărca toți operatorii de comparație “în același timp”

```
#include <compare>

class X {
    // defines ==, !=, <, >, <=, >=, <=>
    friend auto operator<=>(const X&, const X&) = default;
};
```

(sursa: <https://stackoverflow.com/questions/4421706>)

- în aceste cazuri trebuie să definim două funcții de supraîncărcare:
 - $\text{int} + \text{tipClasa}$
 - $\text{tipClasa} + \text{int}$


```

#include <iostream>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {} // needed to construct temporaries
    loc(int lg, int lt) { longitude = lg; latitude = lt; }
    void show() { cout<<longitude<<" "<<latitude<<"\n";}
    loc operator=(loc op2);
    friend loc operator+(loc op1, int op2);
    friend loc operator+(int op1, loc op2);
};
// + is overloaded for loc + int.
loc operator+(loc op1, int op2){
    loc temp;
    temp.longitude = op1.longitude + op2;
    temp.latitude =op1.latitude + op2;
    return temp;}

```

```

// + is overloaded for int + loc.
loc operator+(int op1, loc op2){
    loc temp;
    temp.longitude = op1 + op2.longitude;
    temp.latitude =op1 + op2.latitude;
    return temp;}

```

```

int main(){
    loc ob1(10, 20), ob2(5, 30) , ob3(7, 14);
    ob1.show();
    ob2.show();
    ob3.show();
    ob1 = ob2 + 10; // both of these
    ob3 = 10 + ob2; // are valid
    ob1.show();
    ob3.show();

    return 0;}

```

supraîncărcarea new și delete

- supraîncărcare op. de folosire memorie în mod dinamic pentru cazuri speciale

```
// Allocate an object.
```

```
void *operator new(size_t size){
```

```
    /* Perform allocation. Throw bad_alloc on failure. Constructor called automatically. */
```

```
    return pointer_to_memory;
```

```
}
```

```
// Delete an object.
```

```
void operator delete(void *p){
```

```
    /* Free memory pointed to by p. Destructor called automatically. */
```

```
}
```

- size_t: predefinit
- pentru new: constructorul este chemat automat
- pentru delete: destructorul este chemat automat
- supraîncărcare la nivel de clasă sau globală

```

#include <iostream>
#include <cstdlib>
#include <new>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {}
    loc(int lg, int lt)
        {longitude = lg; latitude = lt;}
    void show() { cout << longitude << " "
cout << latitude << "\n";}
    void *operator new(size_t size);
    void operator delete(void *p);
};
// new overloaded relative to loc.
void *loc::operator new(size_t size){
    void *p;
    cout << "In overloaded new.\n";
    p = malloc(size);
    if(!p) { bad_alloc ba; throw ba; }
    return p;}

```

```

// delete overloaded relative to loc.
void loc::operator delete(void *p){
    cout << "In overloaded delete.\n";
    free(p);
}
int main(){
    loc *p1, *p2;
    try {p1 = new loc (10, 20); }
    catch (bad_alloc xa)
        { cout << "Allocation error for p1.\n"; return 1;}
    try {p2 = new loc (-10, -20); }
    catch (bad_alloc xa)
        { cout << "Allocation error for p2.\n"; return 1;}
    p1->show();
    p2->show();
    delete p1; delete p2;
    return 0; }

```

In overlo
 In overlo
 10 20
 -10 -20
 In overlo
 In overlo

- dacă new sau delete sunt folosiți pentru alt tip de date în program, versiunile originale sunt folosite
- se poate face overload pe new și delete la nivel global
 - se declara în afara oricărei clase
 - pentru new/delete definiți și global și în clasă, cel din clasă e folosit pentru elemente de tipul clasei, și în rest e folosit cel redefinit global

```

#include <iostream>
#include <cstdlib>
#include <new>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {}
    loc(int lg, int lt)
        {longitude = lg;latitude = lt;}
    void show() {cout << longitude << " ";
        cout << latitude << "\n";}
};

```

// Global new

```

void *operator new(size_t size) {
    void *p;
    p = malloc(size);
    if(!p) { bad_alloc ba; throw ba; }
return p;
}

```

// Global delete

```

void operator delete(void *p) { free(p); }
int main(){
    loc *p1, *p2;
    float *f;
    try {p1 = new loc (10, 20); }
    catch (bad_alloc xa) {
        cout << "Allocation error for p1.\n";
        return 1; }
    try {p2 = new loc (-10, -20); }
    catch (bad_alloc xa) {
        cout << "Allocation error for p2.\n";
        return 1; }
    try {
        f = new float; // uses overloaded new, too }
    catch (bad_alloc xa) {
        cout << "Allocation error for f.\n";
        return 1; }
    *f = 10.10F;
    cout << *f << "\n";
    p1->show();
    p2->show();
    delete p1; delete p2; delete f;
return 0; }

```

new și delete pentru array-uri

- facem overload de două ori

```
// Allocate an array of objects.  
void *operator new[](size_t size) {  
    /* Perform allocation. Throw bad_alloc on failure.  
    Constructor for each element called automatically. */  
    return pointer_to_memory;  
}  
  
// Delete an array of objects.  
void operator delete[](void *p) {  
    /* Free memory pointed to by p. Destructors for each  
    element called automatically. */  
}
```

supraîncărcarea []

```
#include <iostream>
using namespace std;
class atype { int a[3];
public:
    atype(int i, int j, int k) { a[0] = i; a[1] = j; a[2] = k; }
    int operator[](int i) { return a[i]; }
};
int main() {
    atype ob(1, 2, 3);
    cout << ob[1]; // displays 2
    return 0;
}
```

- operatorul [] poate fi folosit și la stânga unei atribuirii (obiectul întors este atunci referință)

supraîncărcarea []

```
#include <iostream>
using namespace std;
class atype { int a[3];
public:
    atype(int i, int j, int k) { a[0] = i; a[1] = j; a[2] = k; }
    int &operator[](int i) { return a[i]; }
};
int main() {
    atype ob(1, 2, 3);
    cout << ob[1]; // displays 2
    cout << " ";
    ob[1] = 25; // [] on left of =
    cout << ob[1]; // now displays 25
    return 0; }
```

- putem în acest fel verifica array-urile
- exemplul următor

// A safe array example.

```
#include <iostream>
```

```
#include <cstdlib>
```

```
using namespace std;
```

```
class atype { int a[3];
```

```
public:
```

```
    atype(int i, int j, int k) {a[0] = i; a[1] = j; a[2] = k;}
```

```
    int &operator[](int i);
```

```
};
```

// Provide range checking for atype.

```
int &atype::operator[](int i)
```

```
{
```

```
    if(i<0 || i> 2) { cout << "Boundary Error\n"; exit(1); }
```

```
    return a[i];
```

```
}
```

```
int main() {
```

```
    atype ob(1, 2, 3);
```

```
    cout << ob[1]; // displays 2
```

```
    cout << " ";
```

```
    ob[1] = 25; // [] appears on left
```

```
    cout << ob[1]; // displays 25
```

```
    ob[3] = 44;
```

```
        // generates runtime error, 3 out-of-range
```

```
    return 0; }
```

supraîncărcarea ()

- nu creăm un nou fel de a chema funcții
- definim un mod de a chema funcții cu număr arbitrar de parametri

double operator()(int a, float f, char *s);

O(10, 23.34, "hi");

echivalent cu O.operator()(10, 23.34, "hi");

supraîncărcarea ()

```
#include <iostream>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {}
    loc(int lg, int lt) {longitude = lg;
latitude = lt;}
    void show() {cout << longitude << " ";
cout << latitude << "\n";}
    loc operator+(loc op2);
    loc operator()(int i, int j);
};
// Overload ( ) for loc.
loc loc::operator()(int i, int j) {
    longitude = i; latitude = j;
return *this;
}
```

```
// Overload + for loc.
loc loc::operator+(loc op2) {
    loc temp;
    temp.longitude = op2.longitude + longitude;
    temp.latitude = op2.latitude + latitude; return
temp;
}
int main() { loc ob1(10, 20), ob2(1, 1);
ob1.show();
ob1(7, 8); // can be executed by itself ob1.show();
ob1 = ob2 + ob1(10, 10); // can be used in
expressions
ob1.show();
return 0; }
```

10 20
7 8
11 11

overload pe ->

- operator unar
- obiect->element
 - obiect generează apelul
 - element trebuie să fie accesibil
 - întoarce un pointer către un obiect din clasă

```
#include <iostream>
using namespace std;
class myclass {
    public:
    int i;
    myclass *operator->() {return this;}
};
```

```
int main() {
    myclass ob; ob->i = 10; // same as ob.i
    cout << ob.i << " " << ob->i;
    return 0;
}
```

supraîncărcarea operatorului ,

- operator binar
- ar trebui ignorate toate valorile mai puțin a celui mai din dreapta operand

```

#include <iostream>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {}
    loc(int lg, int lt) {longitude = lg; latitude = lt;
    void show() {cout << longitude << " ";
cout << latitude << "\n";}
    loc operator+(loc op2);
    loc operator,(loc op2);
};
// overload comma for loc
loc loc::operator,(loc op2){
    loc temp;
    temp.longitude = op2.longitude;
    temp.latitude = op2.latitude;
    cout << op2.longitude << " ";
    cout << op2.latitude << "\n";
return temp;
}

// Overload + for loc
loc loc::operator+(loc op2) {
    loc temp;
    temp.longitude = op2.longitude + longitude;
    temp.latitude = op2.latitude + latitude;
return temp; }

int main() {
loc ob1(10, 20), ob2( 5, 30), ob3(1, 1); ob1.show();
ob2.show(); ob3.show();
cout << "\n";
ob1 = (ob1, ob2+ob2, ob3);
ob1.show(); // displays 1 1, the value of ob3
return 0;
}

```

```

10 20
5 30
1 1
10 60
1 1
1 1

```

Perspective

Curs 5

Proiectarea descendentă a claselor. Moștenirea în C++.

- 1 Controlul accesului la clasa de bază.
- 2 Constructori, destructori și moștenire.
- 3 Redefinirea membrilor unei clase de bază într-o clasă derivată.
- 4 Declarații de acces.